

Question Bank — Week 10

Advanced sorting (Lecture 10) — Questions

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice	2
Part B — Short answer and essay	5
Part C — Trace the algorithms	5
Part D — Array state	6
Part E — Complexity analysis	6
Part F — Find and fix the bug	6
Part G — Write the code	7

Answers are in a separate file: [week10-answers.md](#). Levels: **[what]** recall · **[how]** apply · **[why]** explain and justify. The code is that of `dsa/sorting.py` as Lecture 10 shows it: `_merge(a, lo, mid, hi, scratch)` merges the half-open runs `a[lo:mid]` and `a[mid:hi]`, taking from the left on a tie (`<=`); `_partition(a, lo, hi)` is Lomuto's partition of the inclusive range `a[lo..hi]` around `a[hi]`, with the test `a[j] < pivot`; `quick_sort` swaps the chosen pivot to `a[hi]` first, and recurses into the smaller part; `_sift_down(a, index, size)` sifts in the max-heap `a[0:size]`, whose children of `i` are `2i + 1` and `2i + 2`.

Part	Type	Questions
A	Multiple choice (one correct answer of four)	W10-M01 – W10-M22
B	Short answer and essay	W10-E1 – W10-E5
C	Trace the algorithms	W10-T1 – W10-T5
D	Array state — draw every step	W10-S1 – W10-S3
E	Complexity analysis	W10-K1 – W10-K3
F	Find and fix the bug	W10-B1 – W10-B4
G	Write the code — checked by pytest	W10-C1 – W10-C5

Part A — Multiple choice

W10-M01 [what] Which of these is **not** one of the three steps of divide and conquer?

- **a)** divide the problem into smaller problems of the same kind
- **b)** solve the smaller problems recursively
- **c)** combine their answers
- **d)** compare every pair of elements

W10-M02 [how] The recurrence for the number of steps of merge sort on n elements is:

- **a)** $T(n) = T(n/2) + 1$
- **b)** $T(n) = 2T(n/2) + n$
- **c)** $T(n) = T(n - 1) + n$
- **d)** $T(n) = 2T(n/2) + 1$

W10-M03 [how] `_merge` merges two sorted runs of 4 elements each. The largest number of comparisons it can make is:

- **a)** 7
- **b)** 8
- **c)** 4
- **d)** 16

W10-M04 [why] Merge sort is stable because:

- **a)** it splits the array exactly in the middle
- **b)** it is recursive
- **c)** on a tie, `_merge` takes the element from the left run
- **d)** it uses a scratch array

W10-M05 [what] The extra space merge sort needs, beyond the input, is:

- **a)** $O(1)$
- **b)** $O(\log n)$
- **c)** $O(n \log n)$
- **d)** $O(n)$ — the scratch array

W10-M06 [how] How many calls of `_merge` does `merge_sort` make on 8 elements?

- **a)** 3
- **b)** 8
- **c)** 7
- **d)** 4

W10-M07 [how] `merge_sort(list(range(1024)))` — already sorted — makes how many comparisons?

- **a)** 1,023
- **b)** 5,120
- **c)** 10,240
- **d)** 523,776

W10-M08 [what] During Lomuto's partition, with j the next index to look at, the invariant is:

- **a)** $a[\text{lo}:\text{boundary}] < \text{pivot} \leq a[\text{boundary}:j]$
- **b)** $a[\text{lo}:j]$ is sorted
- **c)** $a[\text{boundary}] == \text{pivot}$
- **d)** $a[j:\text{hi}] \geq \text{pivot}$

W10-M09 [how] `_partition(a, 0, 7)` on $a = [6, 3, 9, 1, 8, 2, 7, 4]$ returns:

- **a)** 4
- **b)** 3
- **c)** 0
- **d)** 7

W10-M10 [how] How many comparisons with the pivot does one Lomuto partition of a range of 8 elements make?

- **a)** 8
- **b)** 3
- **c)** 7
- **d)** 28

W10-M11 [why] `quick_sort(list(range(n)), pivot="first")` makes:

- **a)** about $n \log_2 n$ comparisons
- **b)** $n - 1$ comparisons
- **c)** $n(n - 1)/2$ comparisons
- **d)** about $\log_2 n$ comparisons

W10-M12 [why] On an already sorted range, `pivot="median3"` avoids the worst case because:

- **a)** it makes no comparisons
- **b)** it picks a random element
- **c)** it sorts the range first
- **d)** the median of the first, middle and last values is the middle value, which splits the range in half

W10-M13 [why] Quicksort recurses into the **smaller** part and loops on the larger. This bounds:

- **a)** the call stack at $O(\log n)$ frames
- **b)** the running time at $O(n \log n)$
- **c)** the number of swaps at n
- **d)** the stack at $O(1)$ on every input

W10-M14 [what] Quicksort is not stable because:

- **a)** it is recursive
- **b)** its partition swaps elements across long distances
- **c)** it may use a random pivot

- **d)** it needs a scratch array

W10-M15 [what] In the array layout of a heap, the children of index i are at:

- **a)** $i + 1$ and $i + 2$
- **b)** $2i$ and $2i + 1$
- **c)** $2i + 1$ and $2i + 2$
- **d)** $i // 2$ and $i // 2 + 1$

W10-M16 [how] Build-heap on $[4, 10, 3, 5, 1, 8, 2, 7]$ gives:

- **a)** $[10, 8, 7, 5, 4, 3, 2, 1]$
- **b)** $[1, 2, 3, 4, 5, 7, 8, 10]$
- **c)** $[10, 4, 8, 5, 1, 3, 2, 7]$
- **d)** $[10, 7, 8, 5, 1, 3, 2, 4]$

W10-M17 [why] Build-heap is $O(n)$, not $O(n \log n)$, because:

- **a)** most nodes are near the bottom, where a sift-down is short
- **b)** it never swaps
- **c)** only the root is sifted
- **d)** the input is already a heap

W10-M18 [how] For $n = 10$, heap sort's build loop sifts down the indices:

- **a)** 9, 8, ..., 0
- **b)** 5, 4, 3, 2, 1, 0
- **c)** 4, 3, 2, 1, 0
- **d)** 0, 1, 2, 3, 4

W10-M19 [what] Which sort is $O(n \log n)$ in the **worst** case **and** needs only $O(1)$ extra space?

- **a)** merge sort
- **b)** heap sort
- **c)** quicksort
- **d)** insertion sort

W10-M20 [why] The lower bound of about $n \log_2 n$ comparisons applies to:

- **a)** every sorting algorithm
- **b)** comparison sorts only
- **c)** merge sort only
- **d)** counting sort

W10-M21 [what] Python's `sorted()` and `list.sort()` use:

- **a)** quicksort
- **b)** heap sort
- **c)** introsort
- **d)** Timsort

W10-M22 [how] `sorted(list(range(1000)))` — already sorted — makes how many comparisons?

- **a)** 999
 - **b)** about 10,000
 - **c)** 499,500
 - **d)** 0
-

Part B — Short answer and essay

W10-E1 [why] (5 marks) Describe merge sort as divide and conquer. Write its recurrence and solve it with a recursion tree. Give its extra space, and explain what makes it stable.

W10-E2 [why] (4 marks) Describe Lomuto's partition: what it does, its invariant, and why the pivot is in its final place afterwards. Why is there nothing to "combine" in quicksort?

W10-E3 [why] (4 marks) Compare the pivot strategies "first", "random" and "median3": the cost of choosing, and the inputs on which each is at its worst. Why is sorted input the case that matters?

W10-E4 [why] (4 marks) Explain how heap sort uses an array as a tree. Describe build-heap and the sort phase, and argue that build-heap is $O(n)$. Why is heap sort often slower in practice than quicksort, though its worst case is better?

W10-E5 [why] (3 marks) Define a **stable** sort. Give a practical reason to want one, say which of the six sorts in `dsa/sorting.py` are stable, and explain why sorting (key, tag) tuples does **not** test stability.

Part C — Trace the algorithms

W10-T1 [how] Trace `_merge(a, 0, 4, 7, scratch)` on `a = [3, 7, 8, 12, 1, 7, 10]`: for each comparison, give `i`, `j`, `k`, the two values compared and which one is copied. Which leftover loop runs? Which of the two 7s goes first?

W10-T2 [how] Trace `_partition(a, 0, 7)` on `a = [5, 8, 1, 9, 3, 7, 2, 6]` as a table of `j`, `a[j]`, whether it is swapped, the array, and boundary. What does it return?

W10-T3 [how] Trace `quick_sort([6, 3, 9, 1, 8, 2, 7, 4], pivot="last")`: for each partition give the range `lo..hi`, the pivot value, where it lands (`q`), and the array afterwards. In which order are the ranges partitioned, and why?

W10-T4 [how] Trace `heap_sort([4, 10, 3, 5, 1, 8, 2, 7])`: the array after each sift of build-heap, and after each of the first three steps of the sort phase.

W10-T5 [why] How many comparisons does `merge_sort` make on each of these? Explain the differences.

<code>[1, 2, 3, 4, 5, 6, 7, 8]</code>	<code>[5, 6, 7, 8, 1, 2, 3, 4]</code>
<code>[1, 3, 5, 7, 2, 4, 6, 8]</code>	<code>[8, 7, 6, 5, 4, 3, 2, 1]</code>

Part D — Array state

W10-S1 [how] Draw the complete merge tree of `merge_sort([6, 5, 3, 1, 8, 7, 2, 4])`: every split, down to single elements, and every merge back up, with the contents of each run. Number the merges in the order they happen.

W10-S2 [how] Draw every partition of `quick_sort([9, 1, 8, 2, 7, 3, 6, 4, 5], pivot="median3")`: the range, the three candidates and the chosen pivot, the array after the partition, and which part is recursed into and which is looped on.

W10-S3 [how] For `heap_sort([2, 9, 4, 7, 1, 8, 5, 3])`, draw the heap **as a tree** and **as an array** after build-heap, and after each of the first three steps of the sort phase. Mark the sorted tail.

Part E — Complexity analysis

W10-K1 [how] Solve each recurrence (give Θ), and name the algorithm and the case it describes:

- (a) $T(n) = 2T(n/2) + n$
- (b) $T(n) = T(n-1) + (n-1), T(1) = 0$
- (c) $T(n) = T(n/2) + n$

W10-K2 [how] Write the recurrence for the number of comparisons of `quick_sort(list(range(n)), pivot="first")`, and solve it exactly. What is its stack depth with the smaller-side recursion, and without it?

W10-K3 [why] Three ways to find the k largest of n values: (1) sort everything and take the last k ; (2) k passes, each finding and removing the largest remaining; (3) build a max-heap and take the root k times. Give the cost of each, and say which is best when $k = 1$, $k = 10$ and $k = n/2$, for $n = 1,000,000$.

Part F — Find and fix the bug

W10-B1 [why] Part of a student's `_merge`:

```
while i < mid and j < hi:
    if a[i] < a[j]:
        scratch[k] = a[i]
        i += 1
    else:
        scratch[k] = a[j]
```

```
j += 1
k += 1
```

All 37 tests pass. What is wrong, and on which input does it show?

W10-B2 [why] A student's build-heap:

```
for index in range(n // 2):
    _sift_down(a, index, n)
```

Find an input of 5 elements on which the heap sort that uses it returns a wrong answer — without running it. Explain, and fix.

W10-B3 [how] A student's quicksort, inside `quick_sort_steps`:

```
def sort(lo, hi):
    if lo < hi:
        p = _choose_pivot(a, lo, hi, pivot, rng)
        a[p], a[hi] = a[hi], a[p]
        q = _partition(a, lo, hi)
        yield list(a), (q,)
        yield from sort(lo, q - 1)
        yield from sort(q + 1, hi)
```

It passes all 13 quicksort tests. What goes wrong on `quick_sort(list(range(1000)), pivot="first")`, and why do the tests not see it? Fix it.

W10-B4 [how]

```
def sort(lo, hi):
    if hi - lo <= 1:
        return
    mid = (lo + hi) // 2
    sort(lo, mid)
    sort(mid, hi)
    _merge(a, lo, mid, hi, scratch)
    yield list(a), tuple(range(lo, hi))
```

What does `merge_sort([5, 2, 9, 1, 7])` return, and why?

Part G — Write the code

In `practice/week10.py`; check with `pytest tests/test_practice_week10.py -v`. The tests count comparisons (or reads and writes), so a solution that sorts everything fails where the question asks for less. Do not use `sorted`, `list.sort`, `heapq` or `bisect`.

W10-C1 [how] `merge_k_sorted(lists)` — merge k sorted lists in $O(N \log k)$ comparisons. Why is merging them one after another $O(Nk)$?

W10-C2 [why] `count_inversions(values)` — the number of pairs out of order, in $O(n \log n)$. Why does taking an element from the right run count `len(left) - i` inversions at once?

W10-C3 [why] `kth_smallest(values, k)` — quickselect. Why is its expected cost $O(n)$ when quicksort's is $O(n \log n)$?

W10-C4 [how] `sort_colours(values)` — a list of 0s, 1s and 2s, sorted in place in one pass. What is the invariant of the three regions?

W10-C5 [how] `merge_sort_bottom_up(values)` — merge sort with loops and no recursion: merge runs of width 1, 2, 4, ... How many passes, and what happens to the last run of a pass when n is not a power of two?